Grid Features Defense Packet -- XB2IQX

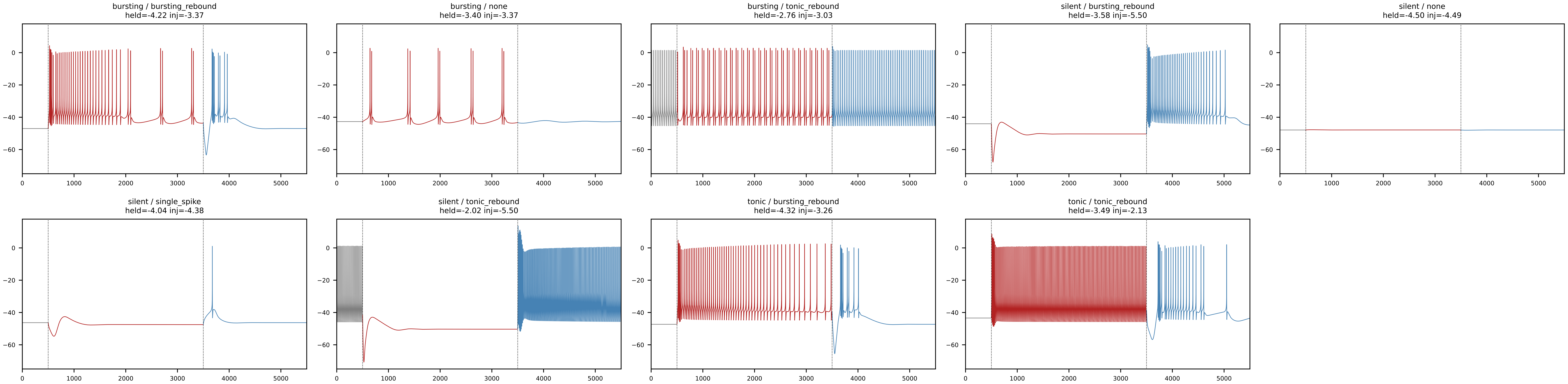
Every trace in this packet is SIMULATED output from a fitted conductance-based single-compartment cell model (src/singlecell_model_v1.py), integrated deterministically at a fixed timestep -- not a raw current-clamp recording. What's being defended here is the feature-extraction methodology applied on top of that model (spike detection, burst/tonic classification, rebound detection, sag depth, adaptation ratio) -- i.e. do these algorithms correctly characterize the dynamics the model actually produces. Every panel below reruns the exact production function on a real simulated trace and marks what it found, so the classification isn't just a text label to take on faith.

`cell_floor = -4.50 nA   |   n grid points = 2500   |   burstiness index = 0.316   |   F-I slope = 18.44 Hz/nA (R^2=0.962)`

Contents:

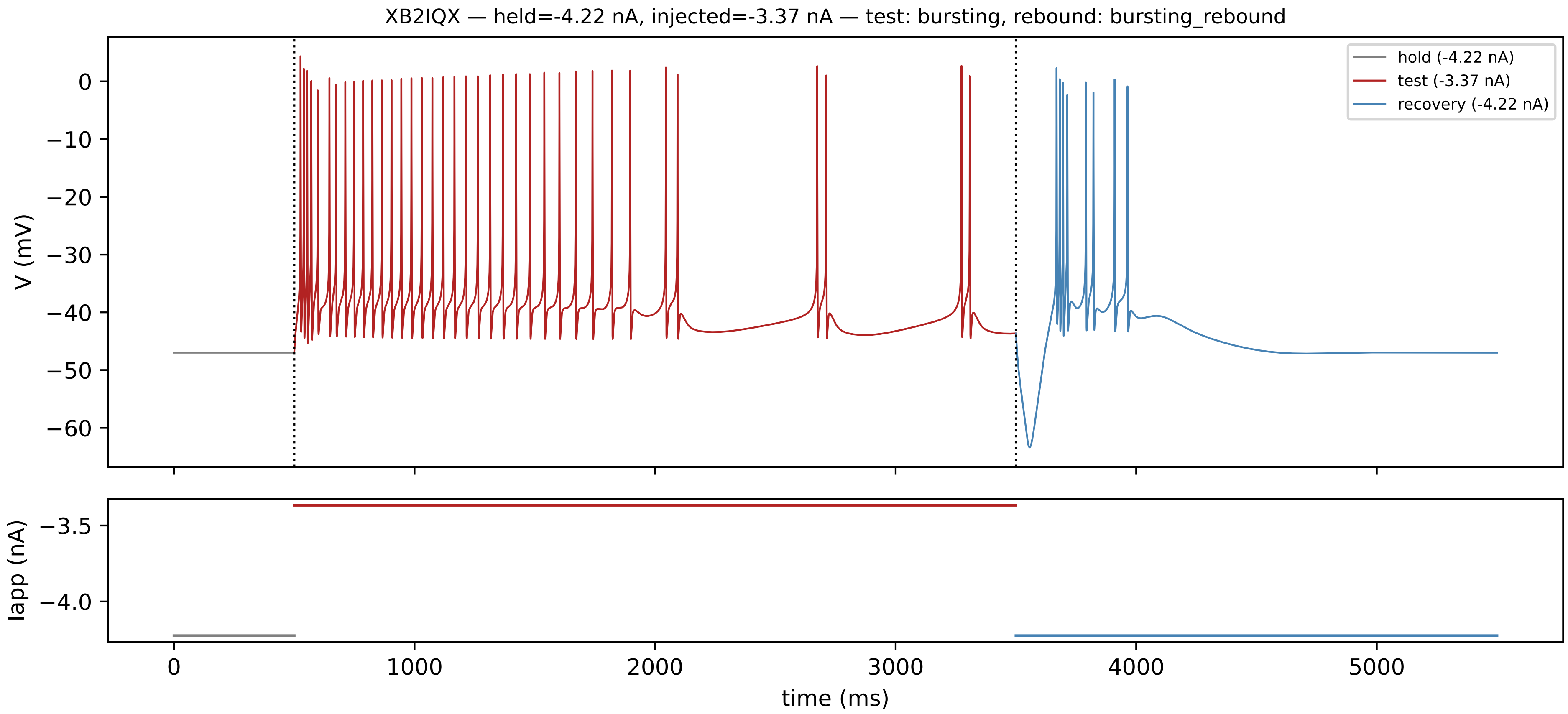

```
python src/generate_defense_packet.py
source: cell_held_injected_grid.pkl + cell_grid_features.pkl
```

1. Representative traces

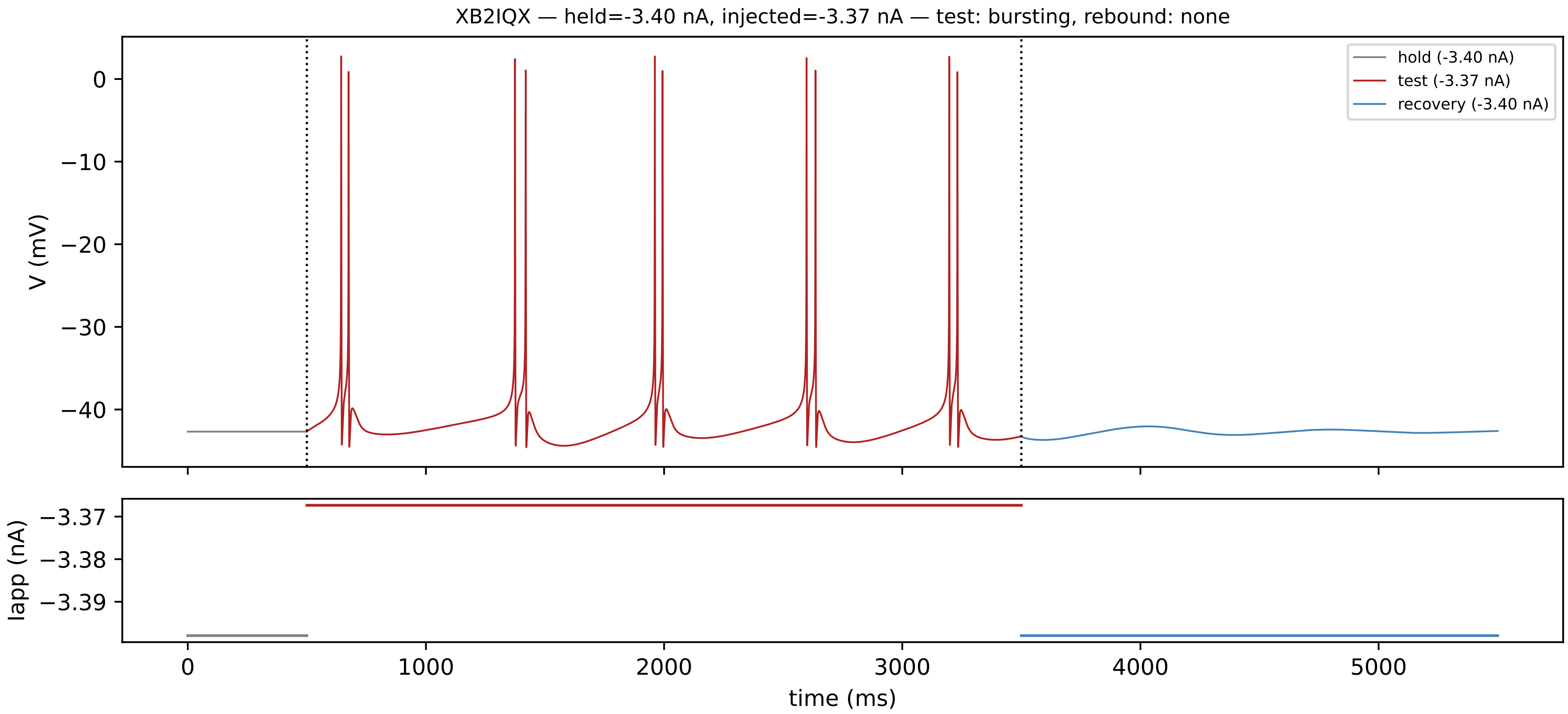


One representative trace per distinct (test-window pattern, rebound pattern) combination actually present across this cell's full 2500-point held x injected grid: gray = held baseline, red = test window (injected current), blue = recovery window (released back to held). Chosen by the same selection plot_example_traces.py uses for its own per-combination figures (most-prominent point, see that script's _prominence_score) -- this is what the cell actually does at each combination, not a hand-picked illustration. Every panel shares the same V (-75 to 18 mV) and time (0-5500 ms) axes, computed from the actual data, so amplitude and duration are directly comparable across panels by eye -- not independently auto-scaled, which would make a subthreshold wobble look as prominent as a real spike train.

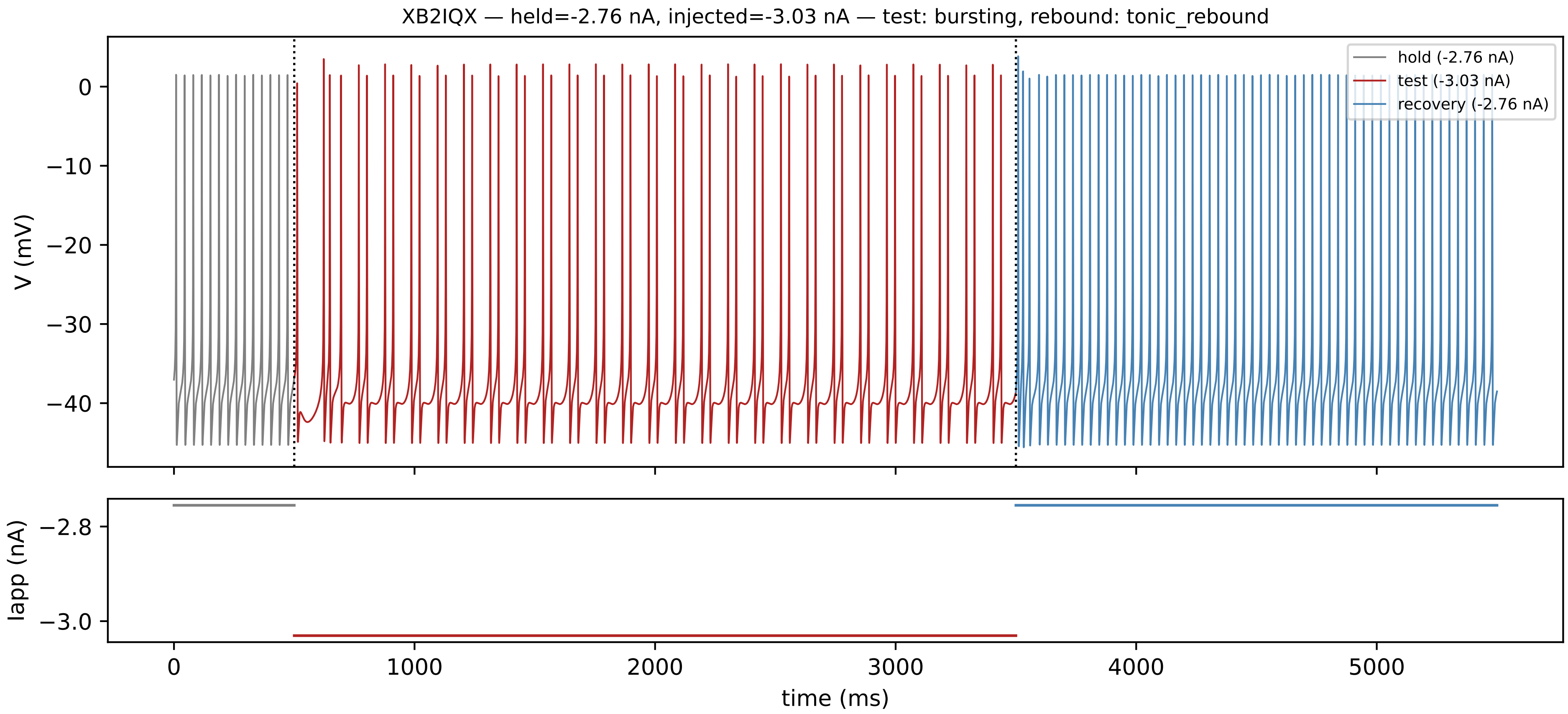
python src/generate_defense_packet.py
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point()



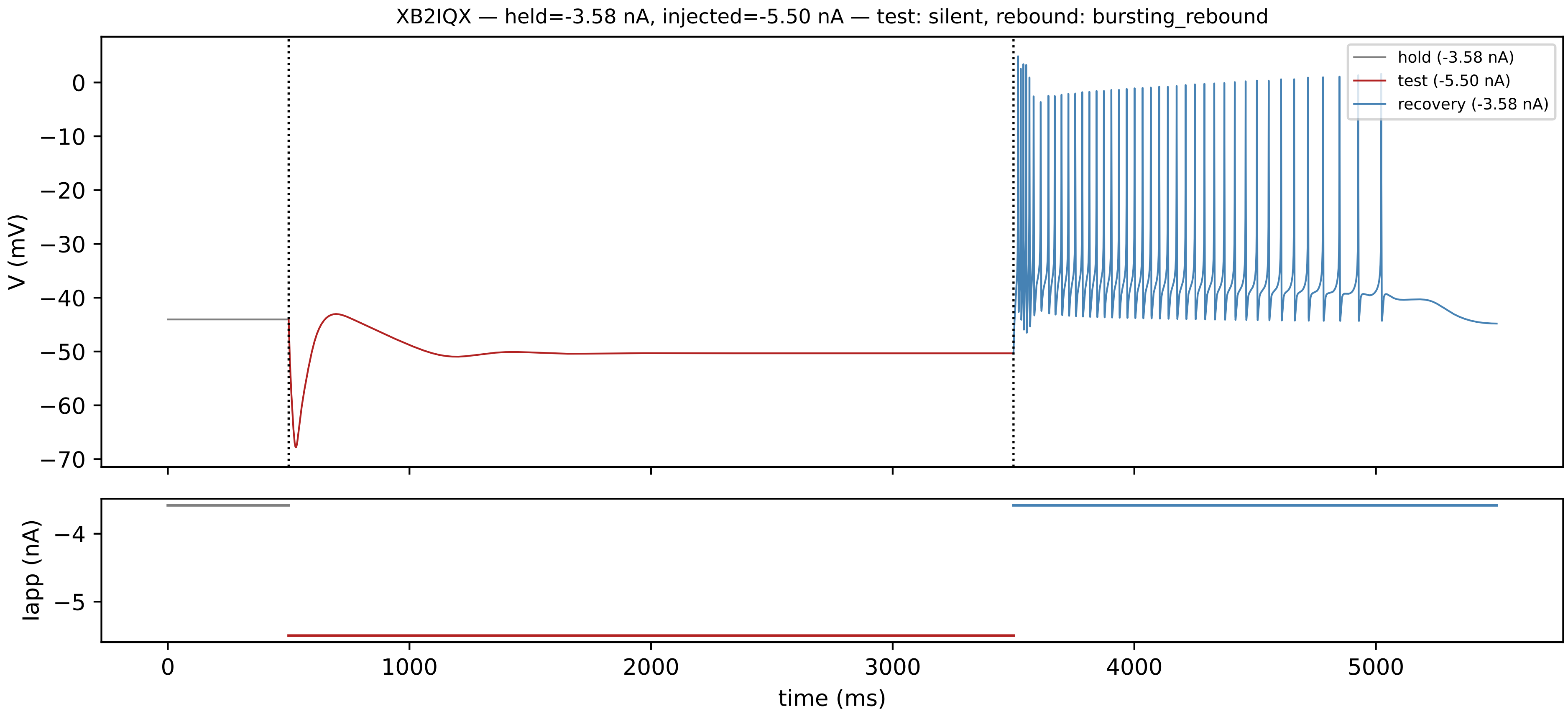
python src/generate_defense_packet.py
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point() -> plot_example_traces.build_example_trace_figure()



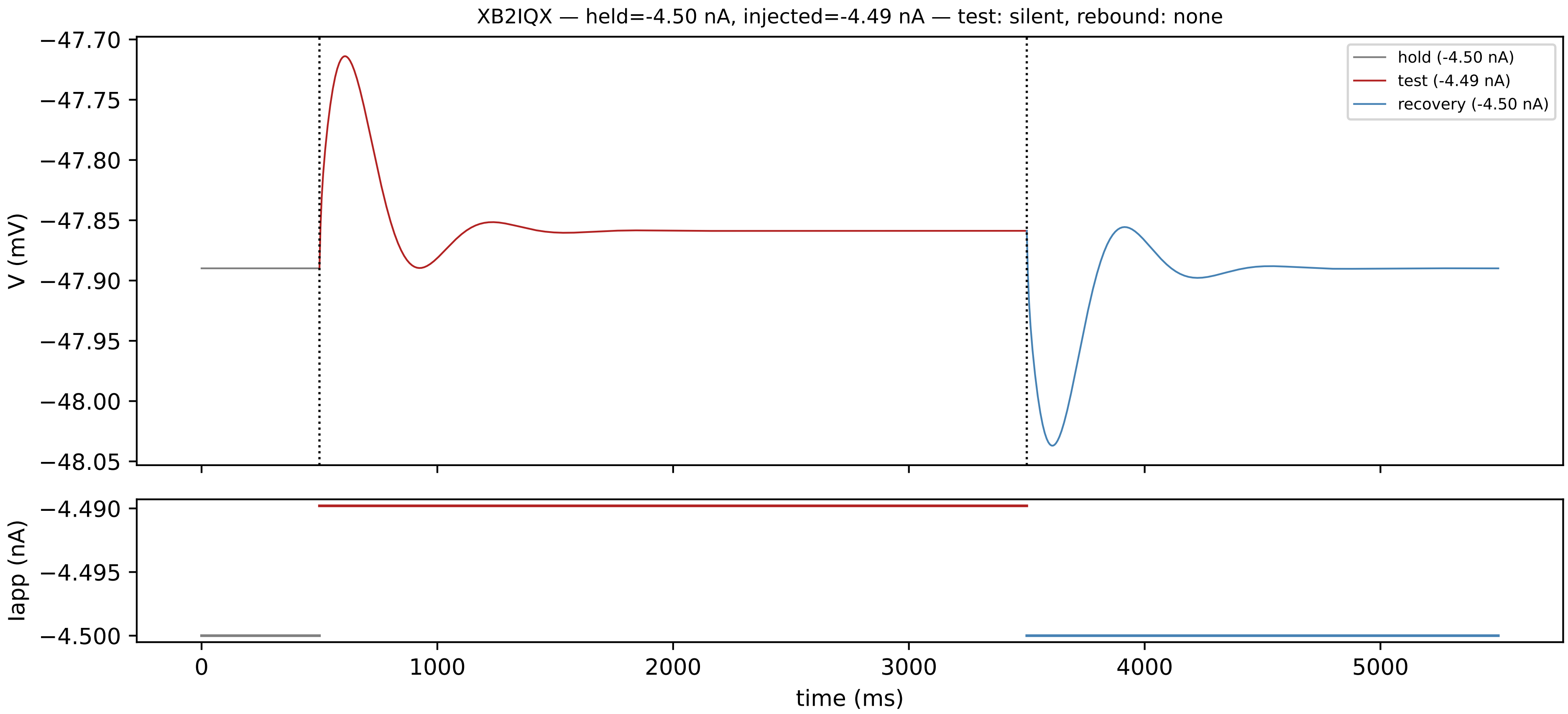
python src/generate_defense_packet.py
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point() -> plot_example_traces.build_example_trace_figure()



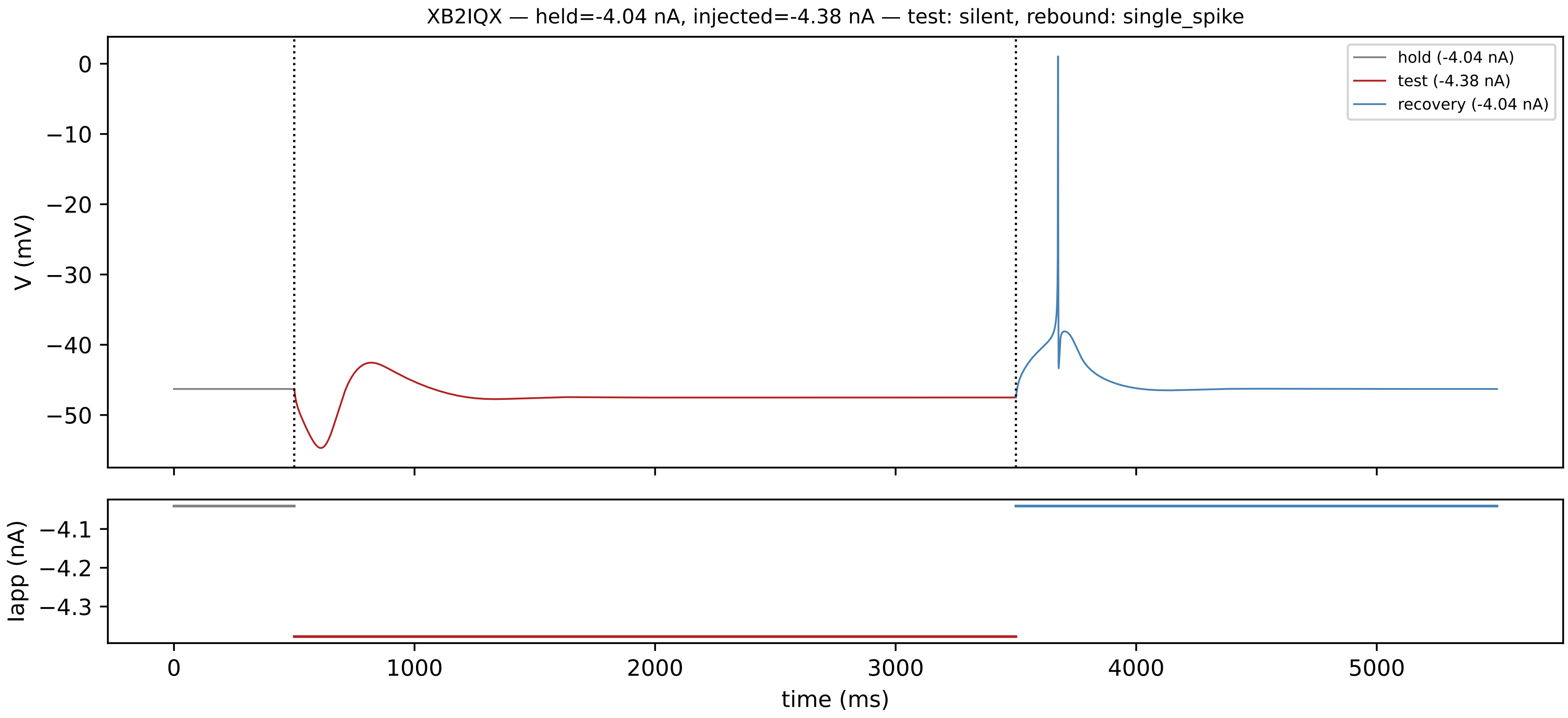
python src/generate_defense_packet.py
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point() -> plot_example_traces.build_example_trace_figure()



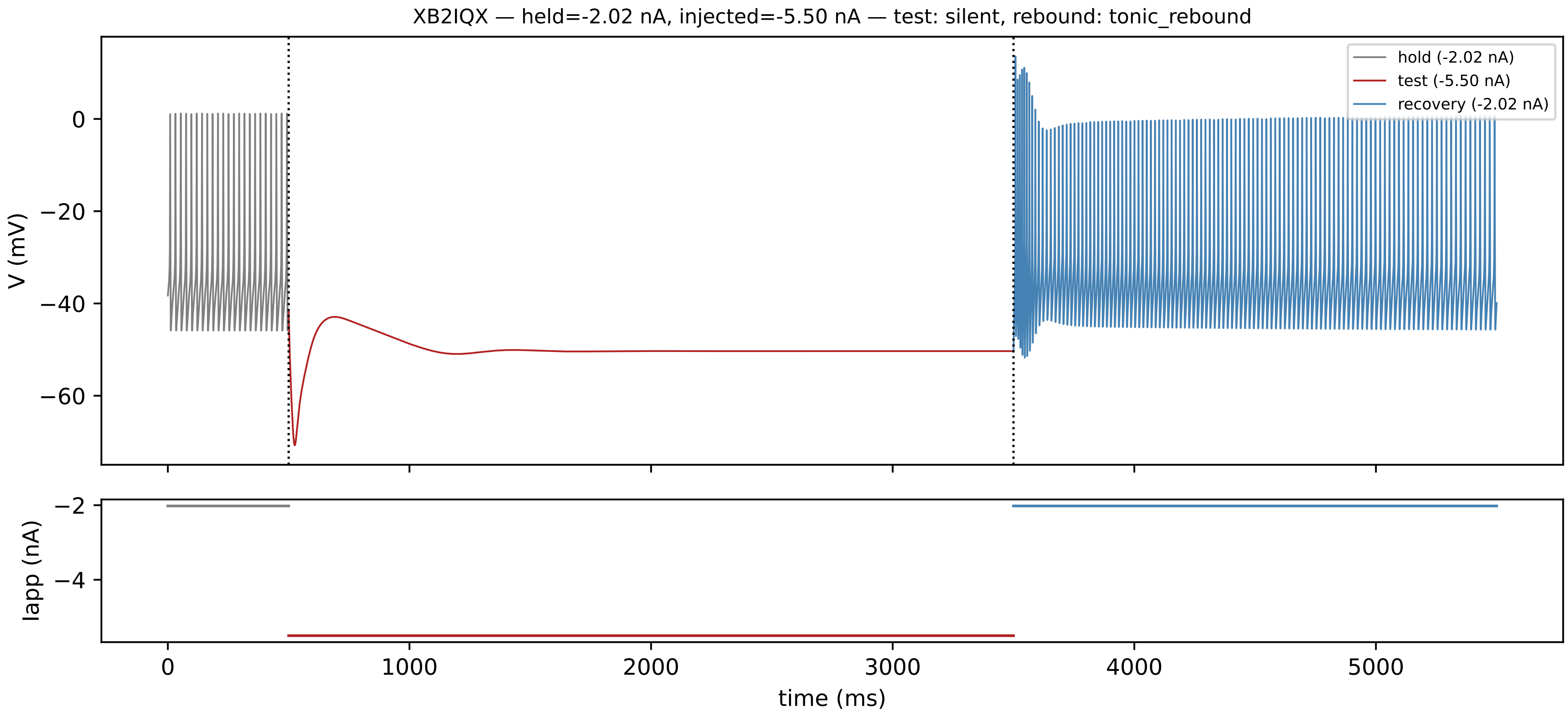
python src/generate_defense_packet.py
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point() -> plot_example_traces.build_example_trace_figure()



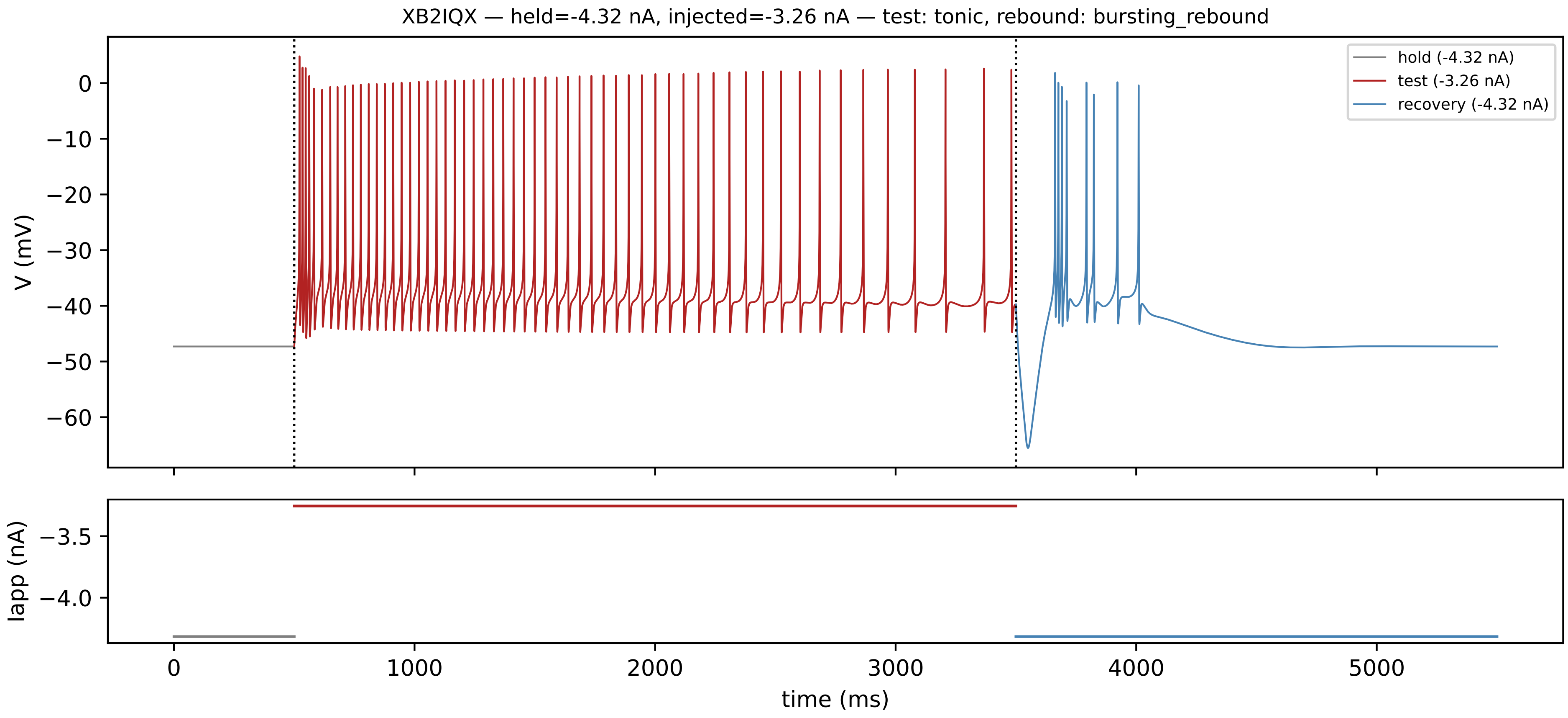
python src/generate_defense_packet.py
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point() -> plot_example_traces.build_example_trace_figure()



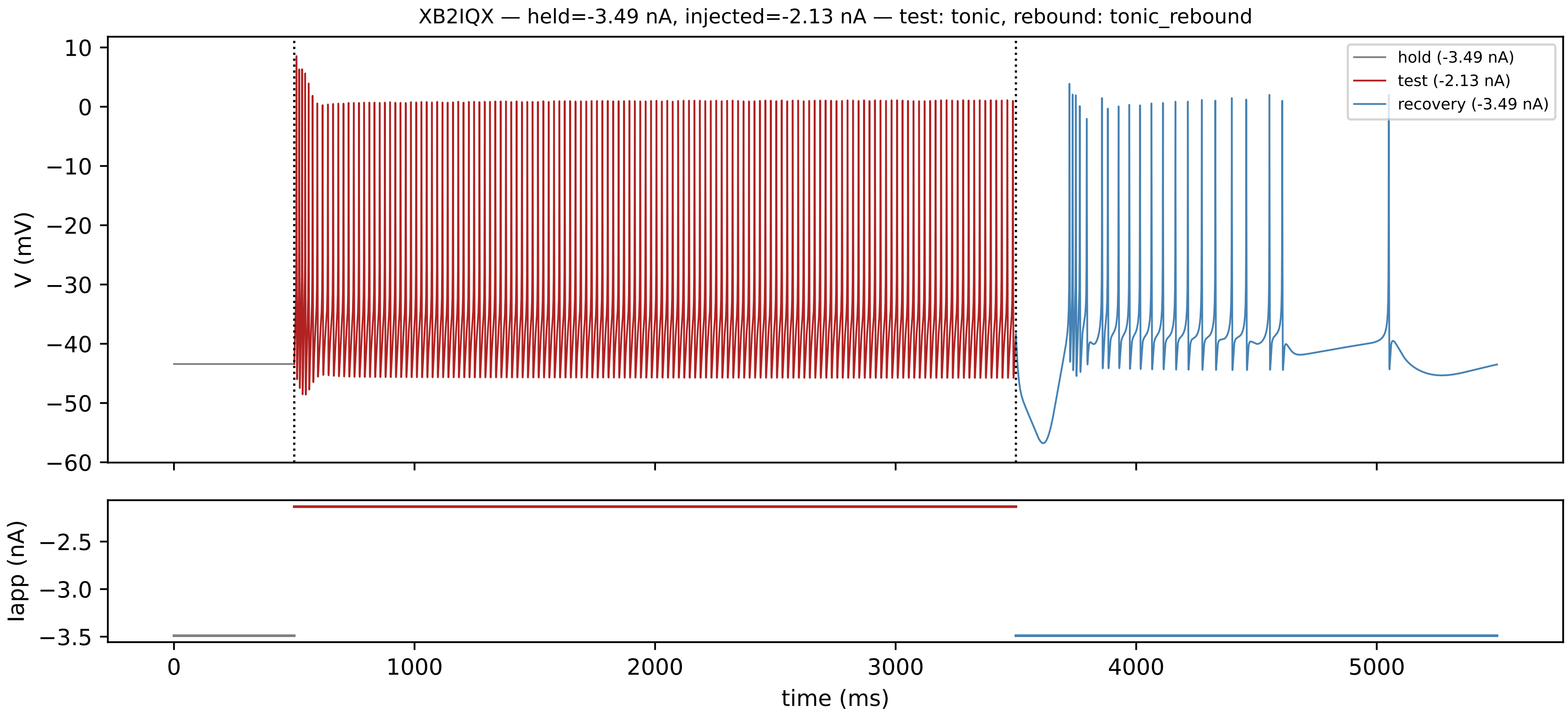
python src/generate_defense_packet.py
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point() -> plot_example_traces.build_example_trace_figure()



python src/generate_defense_packet.py
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point() -> plot_example_traces.build_example_trace_figure()

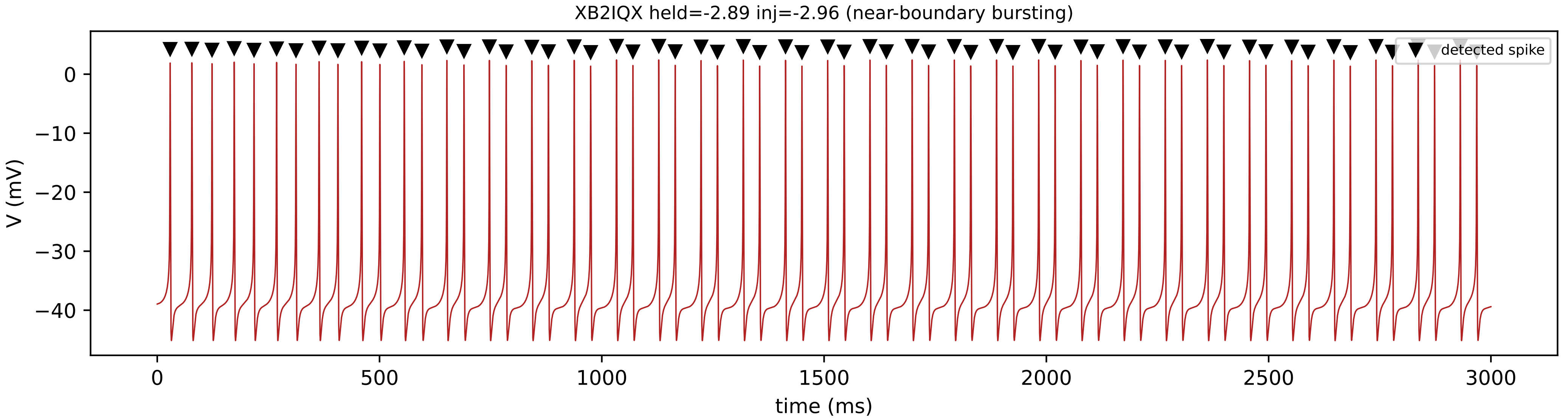


python src/generate_defense_packet.py
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point() -> plot_example_traces.build_example_trace_figure()

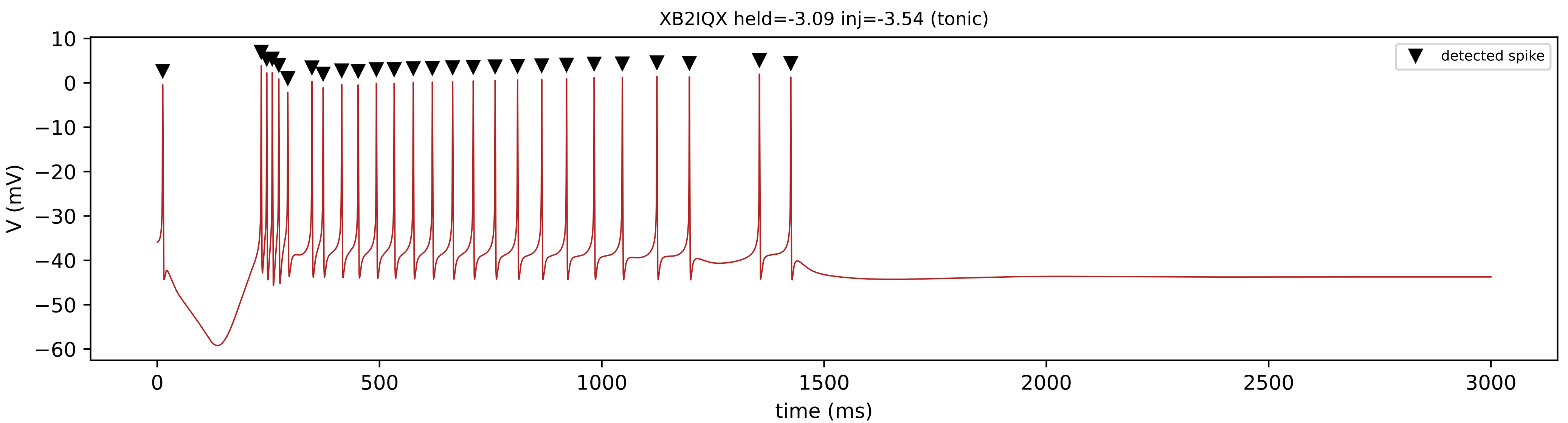


python src/generate_defense_packet.py
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point() -> plot_example_traces.build_example_trace_figure()

2. Spike detection

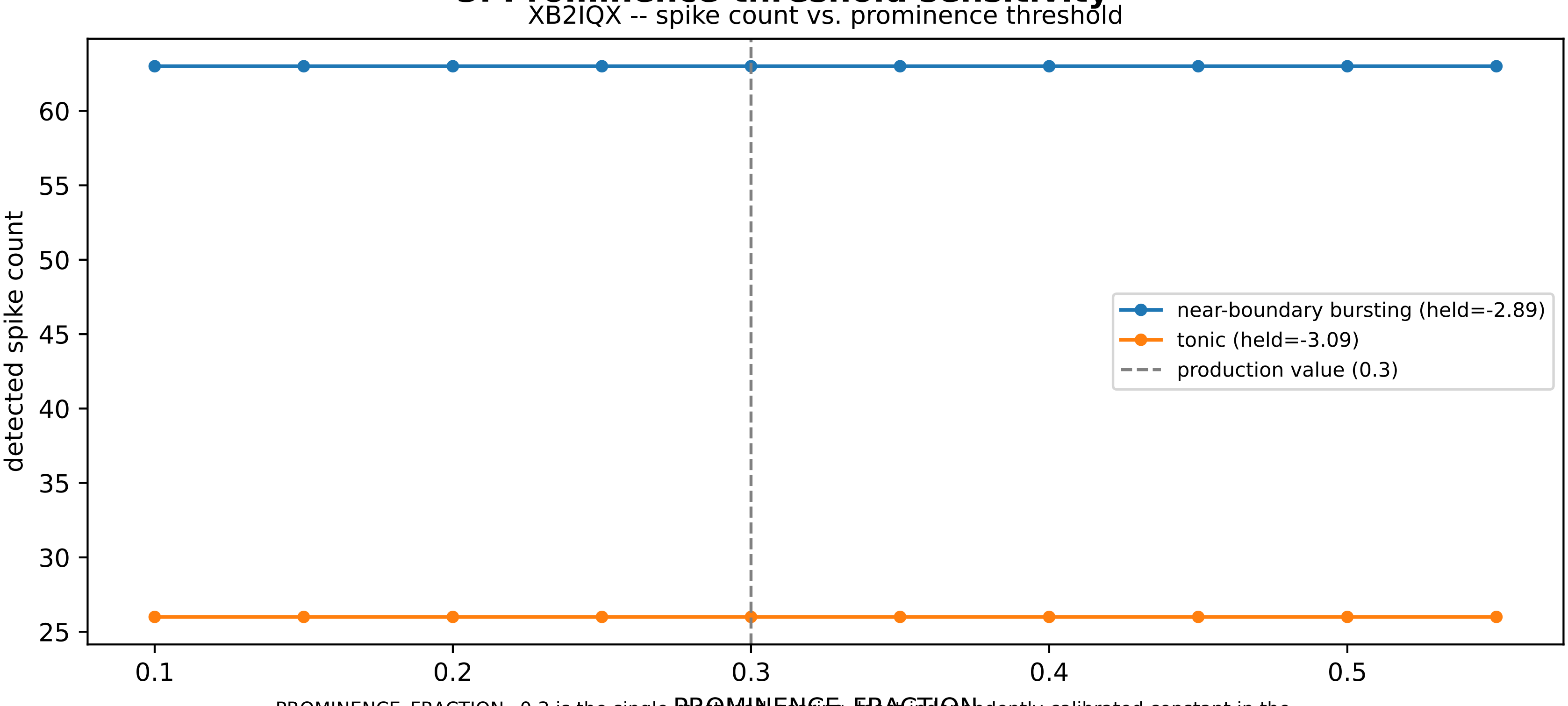


Markers show every peak scipy.signal.find_peaks detects at prominence >= 15.0 mV (the 15mV absolute floor, since 0.30x this trace's own 47.6mV range would be smaller) -- the exact call count_spikes_and_rate/compute_isis_ms use for every spike count, ISI, and burst/tonic classification in the pipeline. 63 spike(s) detected in this window.



python src/generate_defense_packet.py
source: cell_held_injected_grid.pkl -> resimulate_point() -> trace_annotations.mark_spikes() (scipy.signal.find_peaks, same call find_silencing_threshold.compute_isis_ms uses)

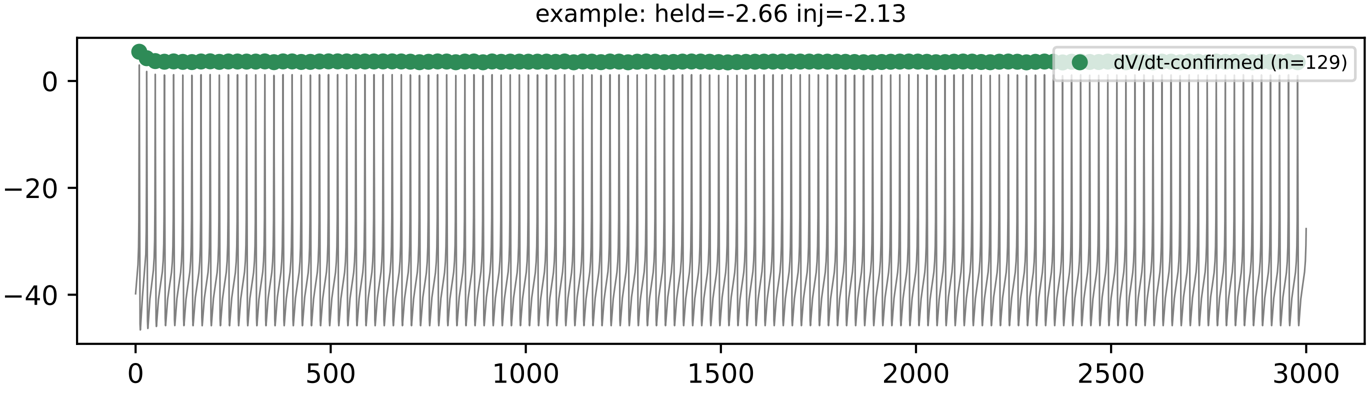
3. Prominence-threshold sensitivity



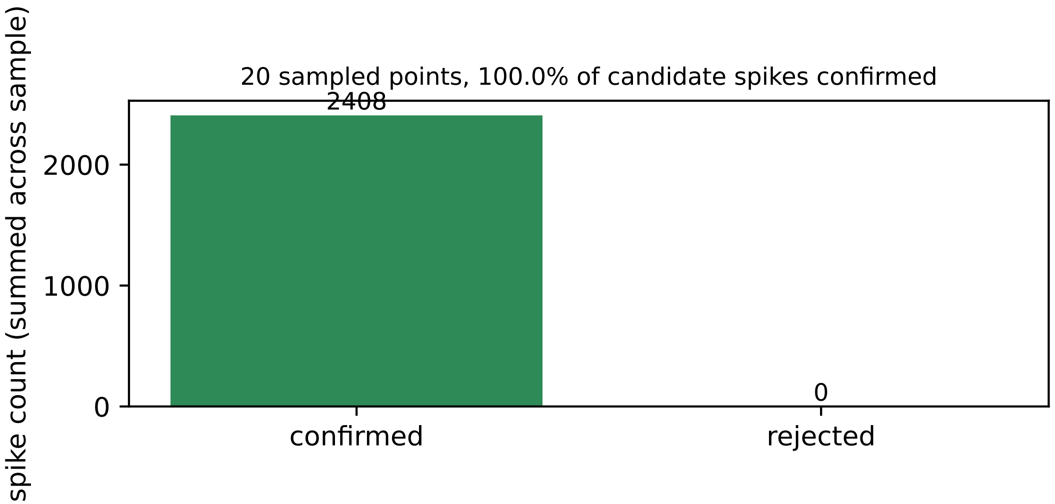
PROMINENCE_FRACTION=0.3 is the single most load-bearing, least-independently-calibrated constant in the pipeline -- every spike count, ISI, and burst/tonic call across the codebase traces back to one find_peaks(..., prominence=max(range*0.3, 15mV)) call (the absolute floor added 2026-08-13, see section 1's representative traces for why). This sweeps the fraction from 0.10 to 0.55 on real, large-amplitude spiking traces (the floor rarely binds here, since these traces' own range x0.3 already exceeds it) -- a flat plateau around the production value means the spike count this pipeline reports is not sensitive to the exact fraction chosen, not an arbitrary cliff-edge pick.

python src/generate_defense_packet.py
source: cell_held_injected_grid.pkl -> resimulate_point() -> scipy.signal.find_peaks swept over PROMINENCE_FRACTION

4. dV/dt cross-validation (unused-in-production detector as a sanity check)



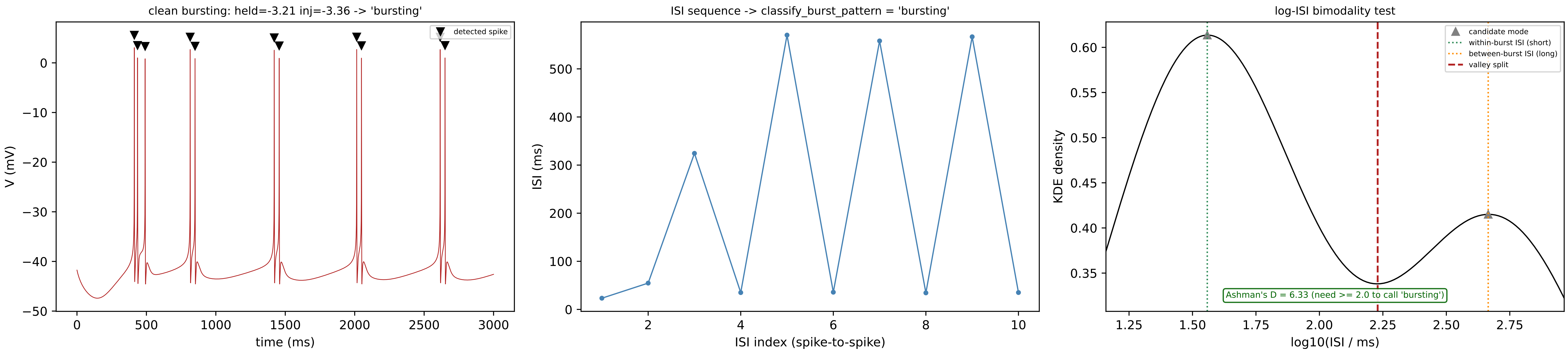
Cross-check against a second, unused-in-production detector: a candidate peak additionally requires a genuine fast upstroke (dV/dt >= 10 mV/ms within 20 ms before the peak, Bean 2007 convention) before counting. 129 of 129 candidate peak(s) confirmed; the two detectors agree on every spike in this window.



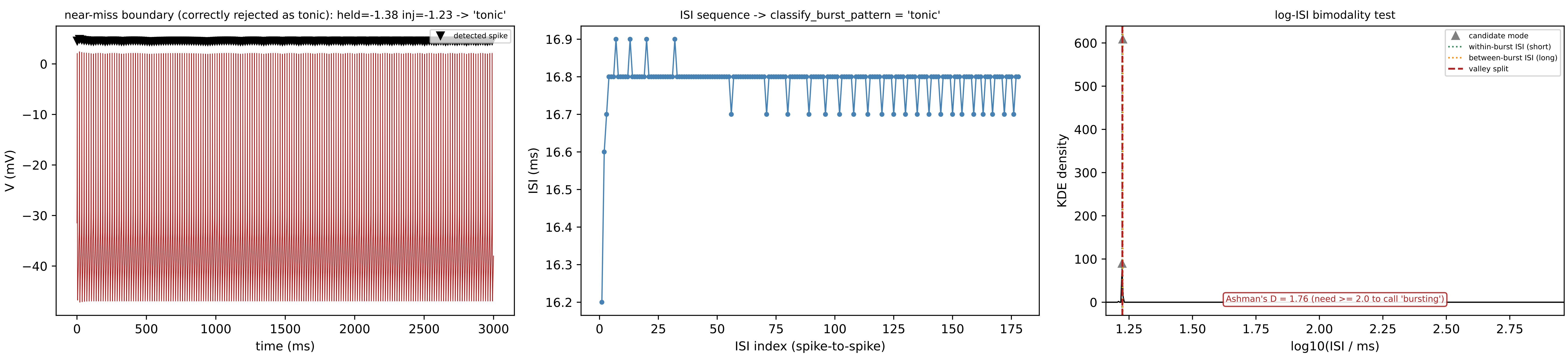
Cross-check across a random sample of 20 XB2IQX grid points: every candidate prominence-based peak is additionally required to pass a dV/dt shape-confirmation gate (detect_spikes_dvdt_confirmed -- present in the codebase but not called by the production pipeline). Agreement close to 100% means the simpler production detector isn't missing a shape check that changes the answer.

python src/generate_defense_packet.py
source: cell_held_injected_grid.pkl -> resimulate_point() -> find_silencing_threshold.detect_spikes_dvdt_confirmed()

5. Tonic / bursting / silent classification



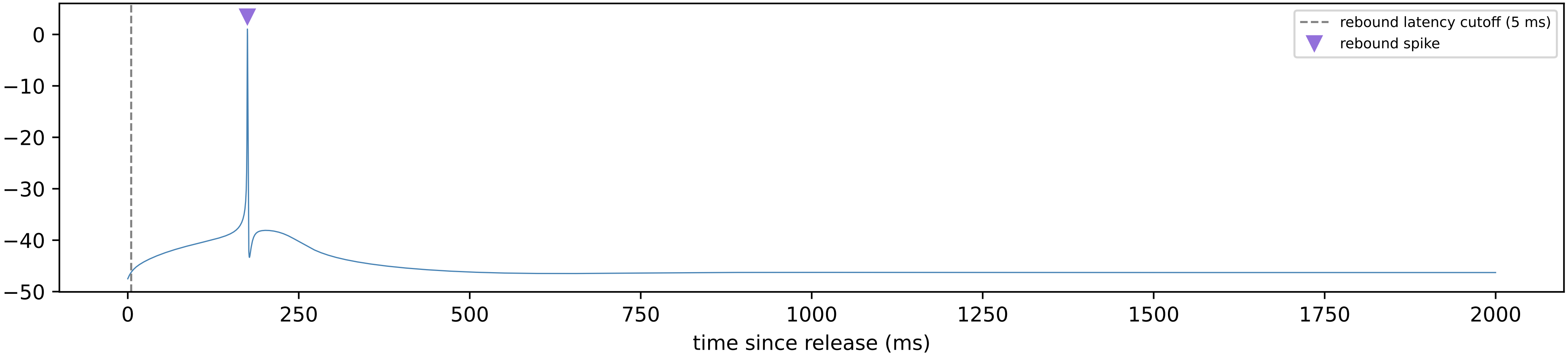
11 spikes, 10 ISIs. Split at 2 candidate mode(s) -> short ISI=36.4 ms, long ISI=504.8 ms (ratio 13.88x, needs >= 1.5x). Avg spikes/burst = 2.20 (needs >= 1.5). Ashman's D = 6.33 (needs >= 2.0). Final call: 'bursting' (bimodality metric / Ashman's D = 6.33).



python src/generate_defense_packet.py
source: cell_held_injected_grid.pkl -> resimulate_point() -> find_silencing_threshold.classify_burst_pattern()

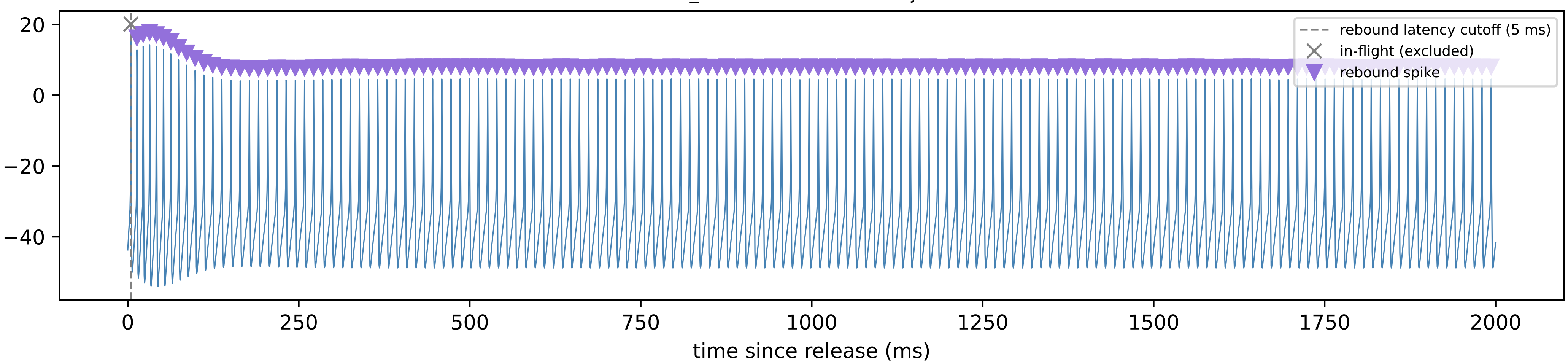
6. Rebound (post-inhibitory) detection

single_spike: held=-4.04 inj=-4.38



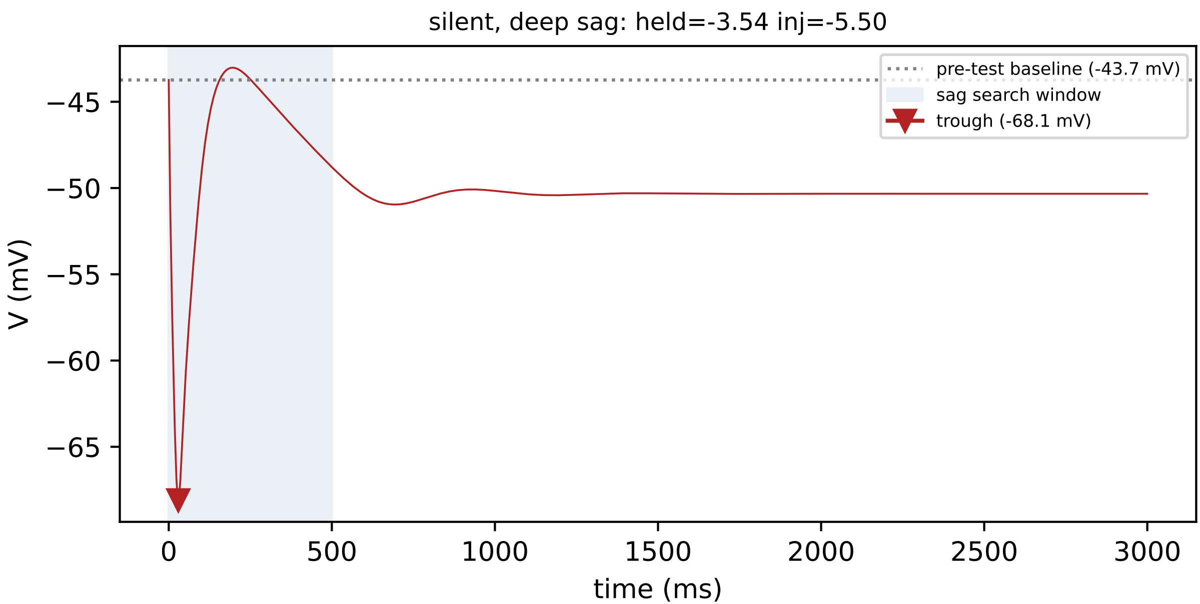
1 spike(s) at or after the 5 ms cutoff count as rebound. rebound_occurred=True.

tonic_rebound: held=-0.32 inj=-3.54



python src/generate_defense_packet.py
source: cell_held_injected_grid.pkl -> resimulate_point() -> trace_annotations.mark_rebound_window() (reproduces run_held_injected_grid.run_test_and_recovery's rebound-peak logic)

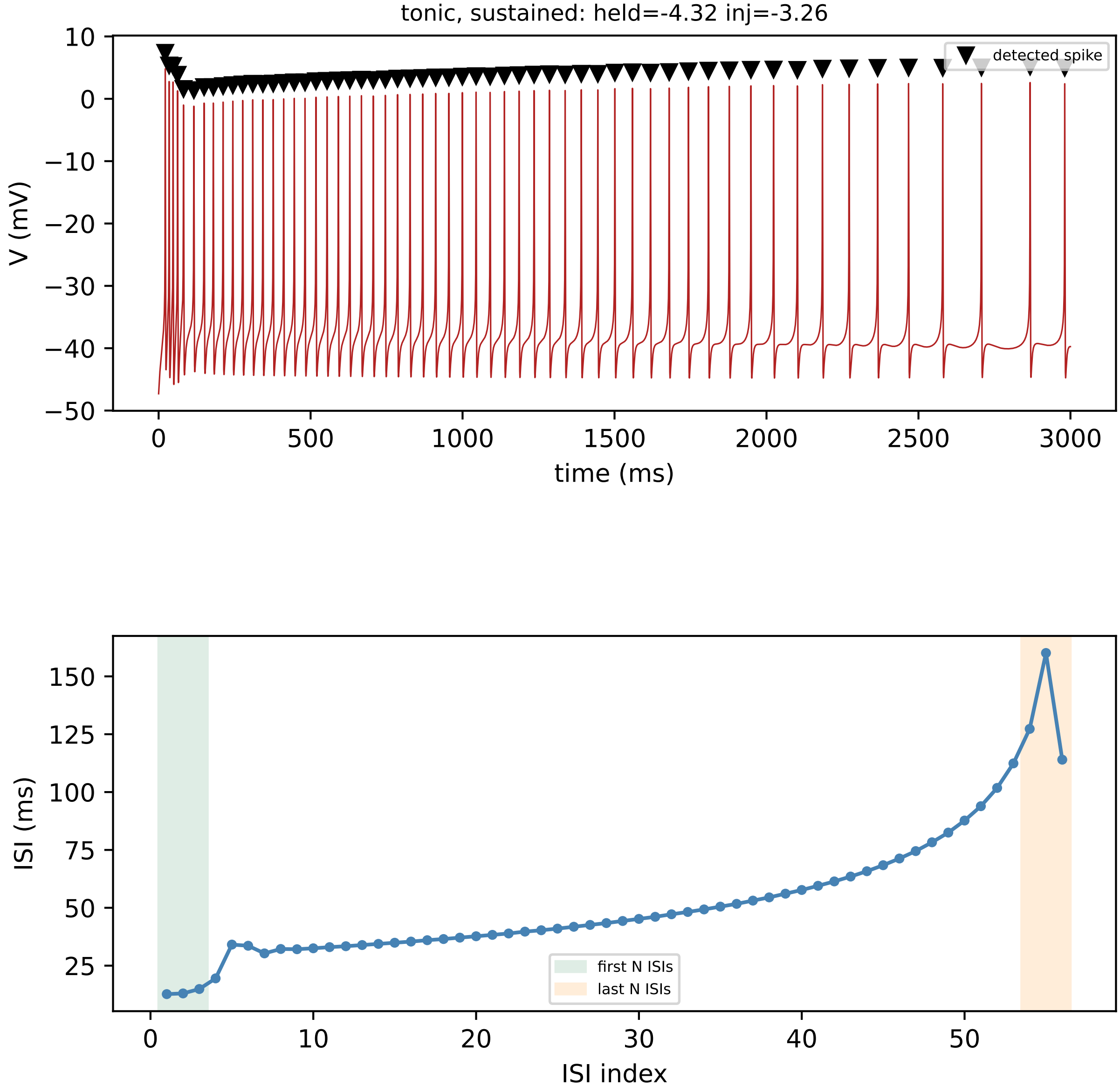
7. Sag depth



Sag depth = baseline - trough = -43.7 - (-68.1) = 24.4 mV. Baseline is the held-current settle's final voltage; trough is the minimum voltage over the full 500 ms window (no spike before it).

python src/generate_defense_packet.py
source: cell_held_injected_grid.pkl -> resimulate_point() -> run_held_injected_grid.compute_pre_spike_sag_trough()

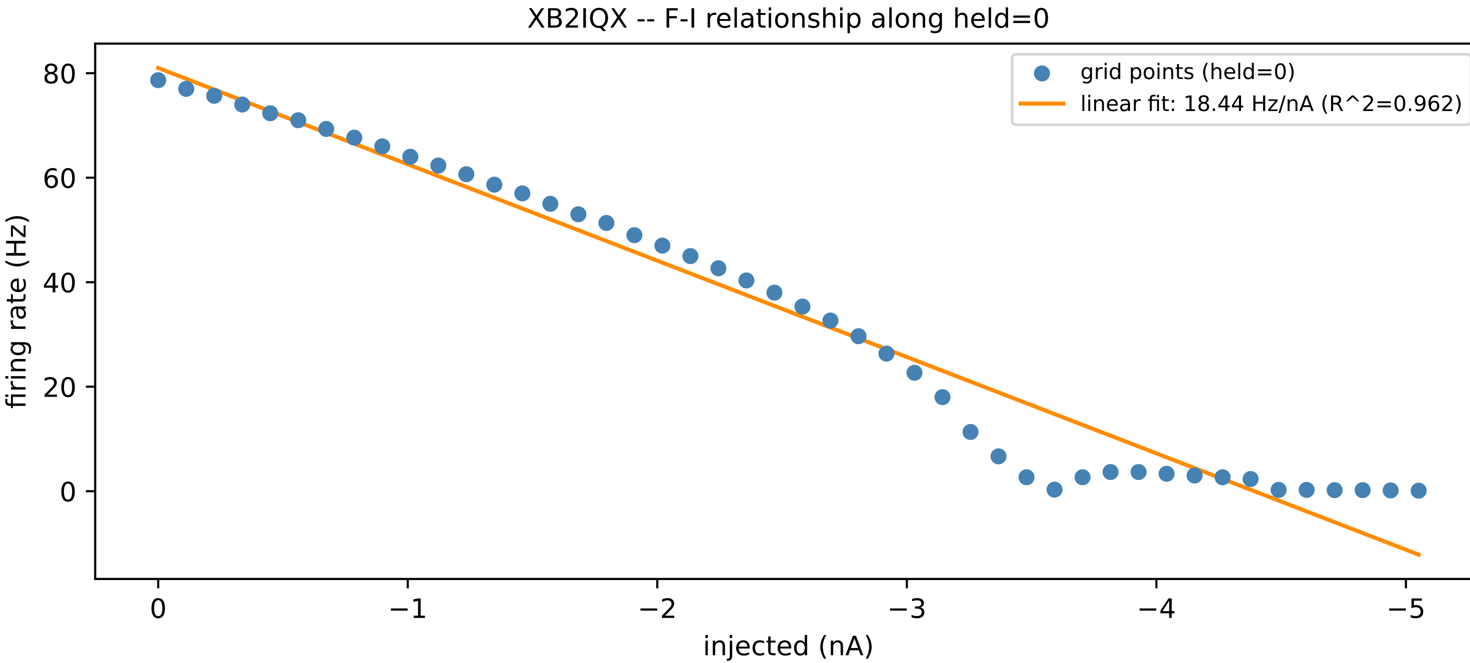
8. Adaptation ratio



Adaptation ratio = mean(last 3 ISIs) / mean(first 3 ISIs) = 9.89. Firing slows (adapts) over the course of this test window.

```
python src/generate_defense_packet.py
source: cell_held_injected_grid.pkl -> resimulate_point() -> run_held_injected_grid.compute_adaptation_ratio()
```

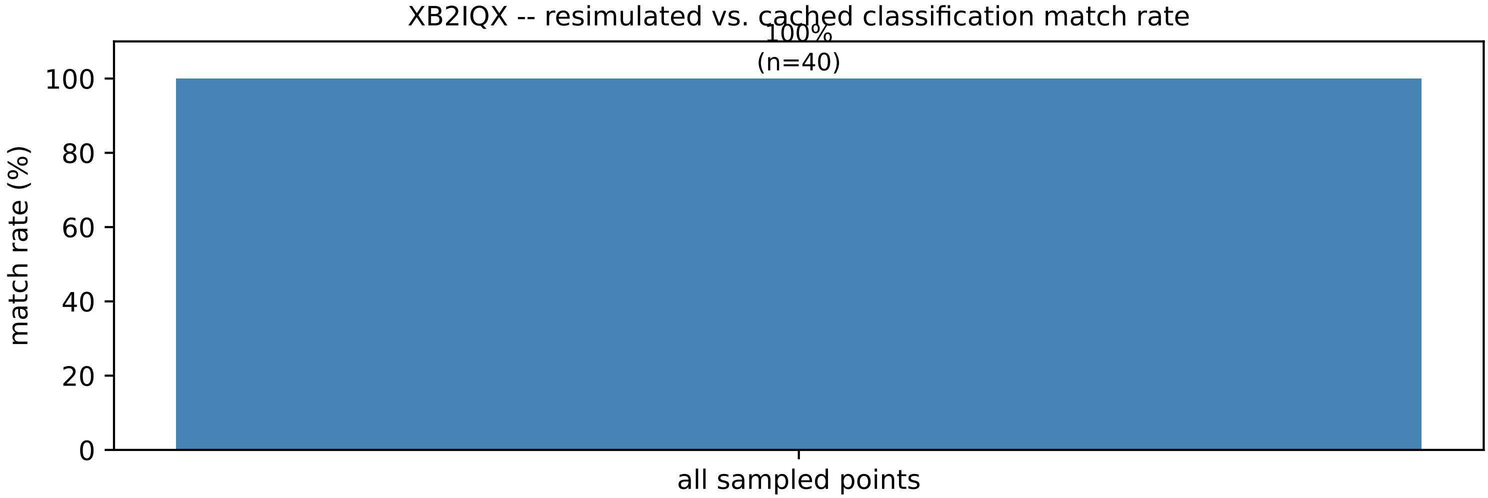
9. Firing rate / F-I slope



46 non-silent, confidently-classified points along the held=0 column (compute_fi_slope's own filter -- flat zero-rate points below rheobase-equivalent excluded so they don't bias the linear fit). This scalar (fi_slope_hz_per_nA) already feeds the cross-cell feature table; this is simply the first time it's been plotted against the data it was fit to.

python src/generate_defense_packet.py
source: cell_held_injected_grid.pkl -> cell_grid_features.pkl (extract_grid_features.compute_fi_slope())

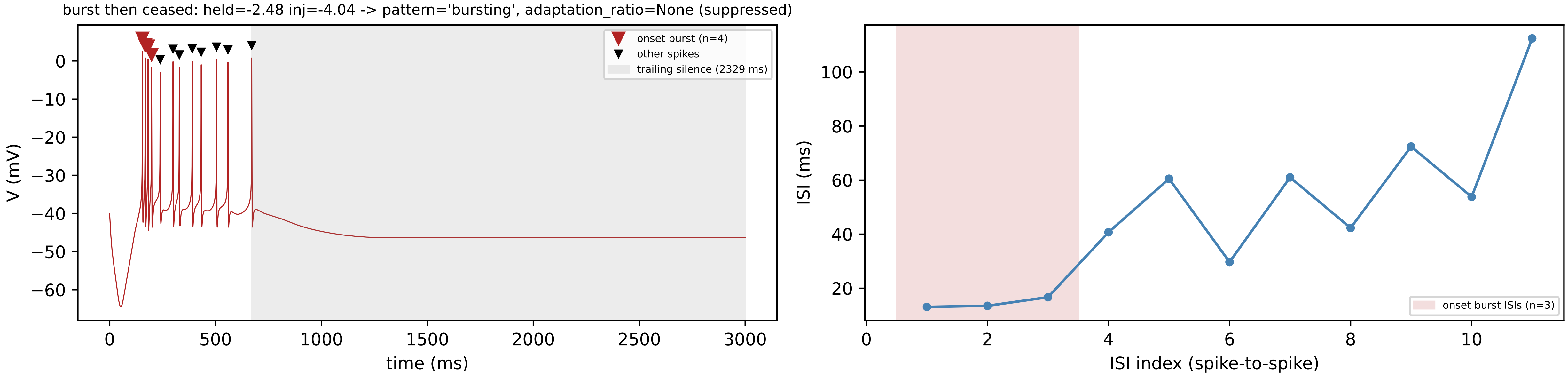
10. Self-consistency check



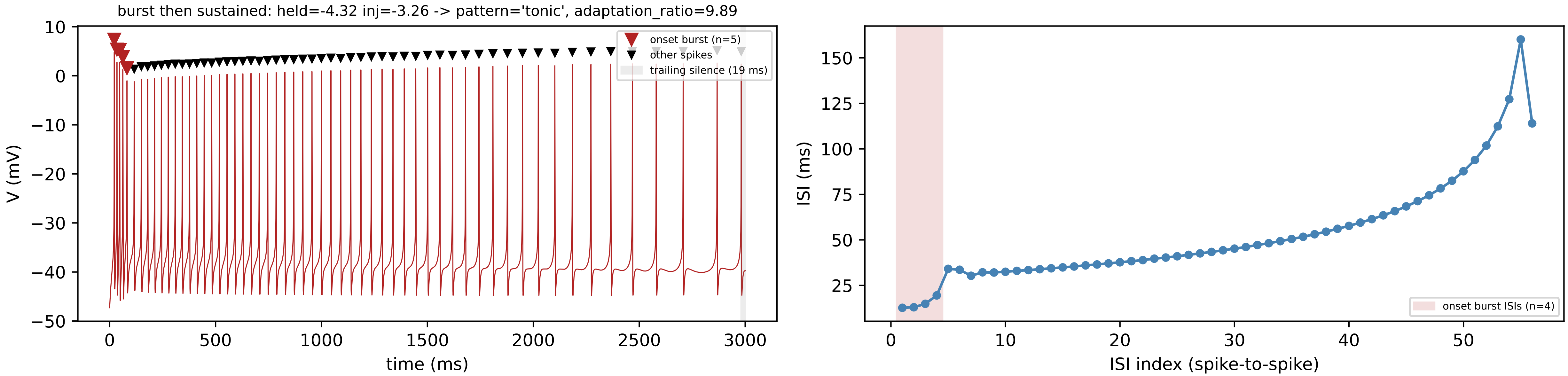
Re-simulates 40 random cached grid points end-to-end and checks whether the resulting test_pattern/rebound_pattern still matches what's stored in cell_held_injected_grid.pkl. Match rate: 100%. Every held level in the current uniform-grid sweep (except held=0, which reuses the cell's own cached limit-cycle state exactly) is settled via a warm-start from whichever already-settled held level is numerically nearest -- the module docstring in run_held_injected_grid.py already documents that this model shows path-dependent (hysteretic) settling near dynamical transitions, so a resimulation that warm-starts from a different already-settled state than the original sweep used can land on a different classification near a boundary. Any mismatches here are consistent with that known property, not a new bug.

```
python src/generate_defense_packet.py
source: cell_held_injected_grid.pkl -> cached test_pattern/rebound_pattern vs. fresh resimulate_point() results
```

11. Burst-onset-then-silence detection

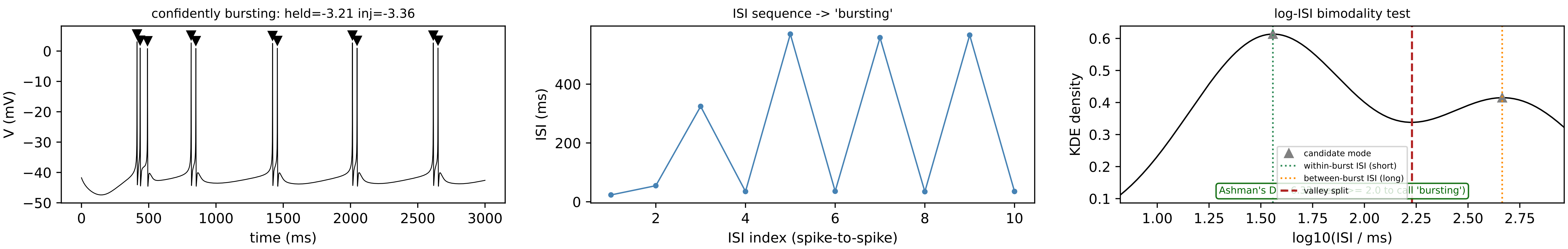


Onset burst detected: 4 spikes (mean ISI 14.4 ms) at window start, found by scanning locally for the first ISI jump -- independent of whatever whole-window tonic/bursting/silent label the KDE bimodality test assigns. Trailing silence after the last spike: 2329 ms of a 3000 ms window. likely_ceased_firing=True -- 2329 ms is >= 3x the most recent ISI, so test_adaptation_ratio is suppressed (reported as None): a first-k/last-k ratio isn't a meaningful 'smooth adaptation' number for a train that stopped partway through the window.

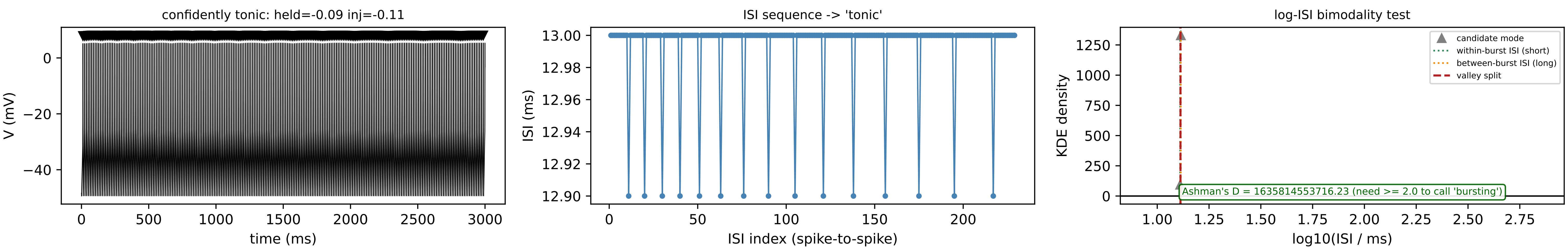


python src/generate_defense_packet.py
source: cell_held_injected_grid.pkl -> resimulate_point() -> run_held_injected_grid.detect_onset_burst()

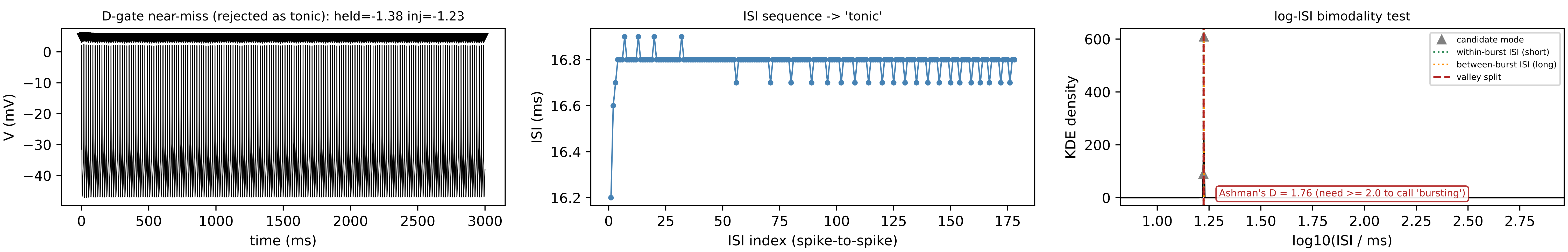
12. Bimodality test: within-burst vs. between-burst ISIs



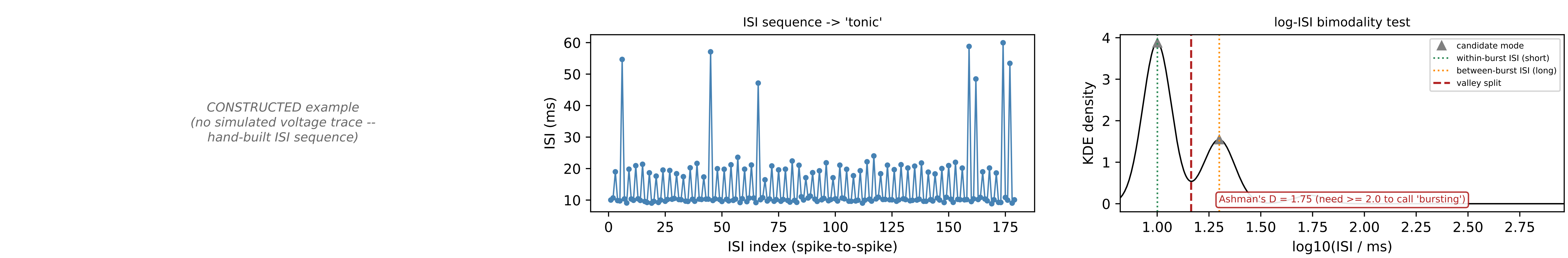
[REAL, resimulated] 11 spikes, 10 ISIs. Split at 2 candidate mode(s) -> short ISI=36.4 ms, long ISI=504.8 ms (ratio 13.88x, needs >= 1.5x). Avg spikes/burst = 2.20 (needs >= 1.5). Ashman's D = 6.33 (needs >= 2.0). Final call: 'bursting' (bimodality metric / Ashman's D = 6.33).



[REAL, resimulated] 230 spikes, 229 ISIs. Split at 2 candidate mode(s) -> short ISI=12.9 ms, long ISI=13.0 ms (ratio 1.01x, needs >= 1.5x). Ashman's D = 1635814553716.23 (needs >= 2.0). Final call: 'tonic' (bimodality metric / Ashman's D = 1635814553716.23).



[REAL, resimulated] 179 spikes, 178 ISIs. Split at 2 candidate mode(s) -> short ISI=16.7 ms, long ISI=16.8 ms (ratio 1.01x, needs >= 1.5x). Ashman's D = 1.76 (needs >= 2.0). Final call: 'tonic' (bimodality metric / Ashman's D = 1.76).



[CONSTRUCTED, not an observed XB2IQX point -- same generator as tests/test_burst_detection.py's Ashman's D boundary test] 180 spikes, 179 ISIs. Split at 2 candidate mode(s) -> short ISI=10.0 ms, long ISI=24.0 ms (ratio 2.40x, needs >= 1.5x). Avg spikes/burst = 3.00 (needs >= 1.5). Ashman's D = 1.75 (needs >= 2.0). Final call: 'tonic' (bimodality metric / Ashman's D = 1.75).

python src/generate_defense_packet.py
source: cell_held_injected_grid.pkl -> resimulate_point() -> find_silencing_threshold.classify_burst_pattern() (4th row: constructed synthetic ISIs, no resimulation)